ChatGPT

Benchmarking GPU-based LLM inference on Kubernetes with vLLM, Triton, and Ray Serve

Executive summary

This report lays out a research-and-engineering path to **benchmark GPU-based LLM inference stacks on Kubernetes** using a single consumer GPU (your **RTX 3060**, single GPU, **6–12GB VRAM unspecified**) and to turn the work into a publishable paper. It focuses on three widely used approaches:

vLLM (standalone): an LLM inference+serving engine built around **PagedAttention** for memory-efficient KV-cache management, designed to increase throughput under batching and long contexts. [1]

NVIDIA Triton Inference Server (now often positioned within **NVIDIA Dynamo / Dynamo-Triton** branding): a general inference server supporting multiple backends, with LLM-specific paths via the **TensorRT-LLM backend** (C++ backend with in-flight batching/paged attention features) and also a **vLLM backend**. [2]

Ray Serve (Ray Serve LLM on Kubernetes via KubeRay): a Kubernetes-native serving/orchestration layer providing an **OpenAI-compatible ingress** and (by default in Ray Serve LLM) standing up **vLLM as the underlying engine**; it adds autoscaling/routing/operational primitives that can matter even on a single GPU. [3]

A key methodological point for the paper: **these are not identical layers of the stack**. vLLM is primarily an *LLM execution engine*; Triton and Ray Serve are *serving platforms* that can host different engines/backends. A rigorous paper should make the comparison explicit by defining at least two comparisons:

Engine-level: vLLM execution vs TensorRT-LLM execution (served via Triton). [4]
Platform-level overhead & operability: vLLM standalone vs vLLM hosted behind Triton vLLM backend vs vLLM hosted behind Ray Serve LLM. [5]

Comparative framing and research questions

What each system "is," at the level you should benchmark

vLLM is designed to improve LLM serving efficiency by reducing KV-cache fragmentation using **PagedAttention** and related scheduling/memory management choices. The vLLM paper reports throughput gains (in their evaluated regimes) while controlling latency, and stresses batching and KV-cache efficiency as bottlenecks in LLM serving. [6]

Triton is a multi-framework inference server (TensorRT / PyTorch / ONNX / Python backends, etc.) that exposes HTTP, gRPC, and Prometheus metrics endpoints (commonly 8000/8001/8002). [7]
For LLMs, Triton provides two relevant "modes" you can benchmark:

- **Triton + TensorRT-LLM backend**: serves engines produced by TensorRT-LLM; the backend documentation highlights in-flight batching and paged attention support in this backend. [8]
- **Triton + vLLM backend**: Triton can embed vLLM as a Python backend; the official tutorial shows a `model.json` controlling `gpu_memory_utilization` and uses a `/generate` endpoint. [9]

Ray Serve LLM deploys an OpenAI-compatible ingress and vLLM-backed deployments; Ray's docs explicitly state that most `vllm serve` engine arguments carry over as `engine_kwargs`, meaning you can keep the execution engine largely comparable while benchmarking Ray's orchestration overhead and scaling behavior. [10]

Research questions suited to a single RTX 3060

On a single GPU, the paper's strongest contribution is usually **a controlled throughput–latency–memory study** rather than distributed scaling. The OSDI'24 work on LLM serving tradeoffs (in the context of vLLM and others) is a useful methodological reference: it varies tail-latency targets and observes capacity/throughput effects. [11]

Concrete, paper-grade questions:

- Under identical prompt/output distributions, how do **p50/p99 latency**, **time-to-first-token**, and **tokens/sec** change with concurrency for each stack? (vLLM vs Triton-vLLM vs RayServe-vLLM vs Triton-TRTLLM) [12]
- How do stacks behave under VRAM pressure (your 6–12GB uncertainty): what are the failure modes and the performance envelopes as you vary `max_model_len` and memory utilization fractions? [13]
- Do platform layers (Triton, Ray) add measurable overhead for short requests, and do they add operational value (metrics, autoscaling knobs, deployment patterns) that could justify the overhead? [14]

Prerequisites, constraints, and model choices for RTX 3060

Skills and tools to learn first

You will implement a reproducible benchmarking pipeline faster if you already have:

- Proficiency with **Linux GPU drivers and debugging** (e.g., verifying `nvidia-smi` works as a first principle check). [15]
- **Container fundamentals** and **NVIDIA Container Toolkit** configuration for Docker and for containerd (Kubernetes). [16]
- Kubernetes basics: **pods**, **deployments**, **services**, **resources/limits**, and GPU scheduling semantics (extended resources). [17]
- Helm usage for Kubernetes packaging (needed for GPU Operator, Triton helm chart, KubeRay operator, vLLM charts). [18]

- Python performance testing: `asyncio`, `httpx`, statistics/plotting (or similar), plus basic experimental design.

In terms of libraries you will likely use:

- vLLM supports an **OpenAI-compatible server** and can be called with the official OpenAI client. [19]
- Triton Python clients can be installed via `pip install tritonclient[all]`, and Triton provides both HTTP and gRPC interfaces. [20]
- Ray Serve on Kubernetes is managed through **RayService** CRs and **KubeRay**. [21]

Hardware/software constraints with 6–12GB VRAM uncertainty

Your RTX 3060 VRAM significantly shapes what you can test:

- For inference, you must budget VRAM for **weights + KV cache + runtime overhead**. Hugging Face's memory guidance for (mixed precision) inference gives a conservative per-parameter estimate (and reminds you there is **activation/overhead** beyond raw weights). [22]
- vLLM exposes explicit knobs to stay within VRAM, especially:
- `--gpu-memory-utilization` (default 0.9) [23]
- `--max-model-len` (can auto-pick a max length that fits) [24]
- `--seed` for reproducibility [25]

Quantization is your main lever for fitting "medium" models:

- vLLM's quantization documentation lists supported approaches including AutoAWQ and BitsAndBytes and notes that quantization trades precision for smaller memory footprint. [26]
- TensorRT-LLM has explicit documentation about numerical precision/quantization recipes. [27]

Suggested small/medium model choices that are realistic on a 3060

The most defensible approach is to benchmark **two tiers**: a "small baseline" that always fits, and a "stress/medium" tier that motivates batching and memory tradeoffs.

Always-fits sanity models (bring-up & debugging)
- `facebook/opt-125m` is explicitly used as a reference model in Triton's "Deploying a vLLM model in Triton" tutorial. [28]
- vLLM Production Stack's minimal example also uses `facebook/opt-125m`, which is useful if you want a Kubernetes-ready baseline. [29]

Small LLM tier (single-GPU friendly)
- `Qwen/Qwen3-0.6B` appears as a vLLM default model and in the official vLLM Docker example. [30]
- vLLM documentation examples also show a 1B-class instruct model (e.g., `unsloth/Llama-3.2-1B-Instruct`) for metrics examples, which is a reasonable "small instruct" benchmark point. [31]

Medium tier (VRAM-sensitive; use quantization or short contexts)
- Ray's Kubernetes example deploys `Qwen/Qwen2.5-7B-Instruct`, which is realistic only if your VRAM is on the higher end and/or you quantize and cap context length. [32]
- For TensorRT-LLM engagement, NVIDIA's examples frequently start from smaller models like `gpt2-`

`medium` in the TensorRT-LLM backend quick start. [33]
- TensorRT-LLM's Qwen3 example documentation lists a range including **0.6B, 1.7B, 4B, 8B** variants; this provides a vendor-aligned path if you want a consistent family across vLLM and TensorRT-LLM (but you should start small on 3060). [34]

Kubernetes GPU setup on a laptop

This section gives a practical baseline you can implement without assuming OS/VRAM details beyond "single laptop GPU." Where behavior differs (Docker vs containerd, minikube vs microk8s vs k3s), you get a concrete path for each.

GPU scheduling essentials in Kubernetes

- GPUs are typically advertised to Kubernetes as an **extended resource** (commonly `nvidia.com/gpu`) via a device plugin. Kubernetes' official "Schedule GPUs" documentation clarifies that you can specify GPU **limits without requests** (limit becomes the request), and if you specify both they must match. [35]
- The NVIDIA Kubernetes device plugin is deployed as a DaemonSet and is responsible for exposing GPU capacity to the cluster. [36]

GPU container enabling: drivers, runtime, and toolkit

Your foundation is:

1) **NVIDIA GPU driver** installed on the host; validate with `nvidia-smi`. This "check first" pattern appears in multiple GPU-on-Kubernetes environment guides (minikube, microk8s). [15]

2) A working container runtime that can mount GPUs into containers. The **NVIDIA Container Toolkit** docs provide explicit runtime configuration steps for Docker and for containerd using `nvidia-ctk`. [16]

3) Optional: **NVIDIA GPU Operator** if you prefer an operator-managed stack (drivers + runtime + device plugin + DCGM telemetry). NVIDIA's docs describe this scope and provide Helm install recipes (including the case where drivers/toolkit are already installed and you disable operator-managed installs). [37]

Local Kubernetes options with GPUs: minikube, microk8s, k3s, kind

minikube

minikube has a maintained tutorial for NVIDIA GPUs, including a Docker-driver path that configures Docker with `nvidia-ctk` and starts minikube with `--gpus all` (or CDI). [38]
It also documents a KVM GPU passthrough path (`--kvm-gpu`) that is substantially more complex and requires a "spare" GPU passthrough setup. [39]

If you already use Docker Desktop / Docker Engine heavily, the **Docker-driver path** is usually the fastest bring-up on a single-dev machine (but you should expect occasional driver/runtime friction). [40]

microk8s

microk8s provides a first-class **GPU add-on** that explicitly states it uses the **NVIDIA GPU Operator**, configures `nvidia-container-runtime` for containerd, and installs the `nvidia.com/gpu` device plugin. [41]
For a laptop with a single GPU, this can be the most straightforward "Kubernetes with GPUs" experience if your host OS and drivers are compatible.

k3s

k3s documents NVIDIA runtime support: install NVIDIA container runtime packages, restart k3s, confirm containerd config includes NVIDIA runtimes, and request the NVIDIA runtime via `runtimeClassName: nvidia` plus `nvidia.com/gpu` limits. [42]
This is a strong choice if you want a lightweight, mostly-containerd-native Kubernetes distribution.

kind

kind is excellent for Kubernetes testing, and Ray's KubeRay quickstart uses kind for creating a local cluster. [43]
However, **GPU passthrough for kind is not a first-class mainstream path**; community discussions note friction when trying to use NVIDIA GPUs inside kind. For a paper whose goal is GPU inference benchmarking (not kind debugging), kind is usually the least time-efficient option unless you already have a known-good recipe. [44]

Recommended "minimum viable GPU-Kubernetes" recipes

Pick one baseline and stick to it for the entire paper to reduce confounds.

Option A: minikube with Docker driver (portable, minimal operator overhead)
Follow minikube's NVIDIA GPU tutorial: configure Docker with `nvidia-ctk runtime configure --runtime=docker` and restart Docker, then `minikube start --driver docker --container-runtime docker --gpus all`. [40]

Option B: microk8s with GPU addon (operator-managed, integrated approach)
Install GPU drivers, verify `nvidia-smi`, then `microk8s enable gpu`. microk8s documents that this flow automatically handles runtime + device plugin configuration via the GPU addon. [45]

Option C: k3s + runtimeClassName: nvidia (lightweight, explicit config)
Use k3s advanced docs: install NVIDIA runtime packages, ensure k3s sees the runtime, and set `runtimeClassName: nvidia` plus `nvidia.com/gpu` in limits. [42]

Kubernetes deployments for vLLM, Triton, and Ray Serve

The deployment goal is the same across systems: **one GPU, one model, stable endpoints, stable metrics, pinned versions**.

vLLM on Kubernetes

You should deploy vLLM in one of two ways: a direct Deployment manifest (simplest) or Helm (more repeatable once configured).

Container image and serving interface

- vLLM provides an official Docker image for the OpenAI-compatible server and shows a Docker run example that mounts a Hugging Face cache and uses `--ipc=host`. [46]
- vLLM's OpenAI-compatible server documentation states it implements OpenAI's APIs and can be called with the official OpenAI client. [19]
- vLLM exposes Prometheus metrics at `/metrics` on the OpenAI server. [47]

Minimal Kubernetes manifest (recommended baseline for a paper)

Key design choices:

- Request 1 GPU via `resources.limits.nvidia.com/gpu: 1` (and optionally requests); follow Kubernetes GPU scheduling rules. [35]
- Constrain VRAM by explicitly setting `--gpu-memory-utilization` and possibly `--max-model-len` to avoid OOM on 6GB variants. [48]
- Pin the image tag (and ideally digest). Docker Hub lists tags for `vllm/vllm-openai`. [49]

Example (edit model and image tag as needed; keep as one file in your repo, e.g., `k8s/vllm-deploy.yaml`):

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vllm
  labels:
    app: vllm
spec:
  replicas: 1
  selector:
    matchLabels:
      app: vllm
  template:
    metadata:
      labels:
        app: vllm
    spec:
      containers:
        - name: vllm
          image: vllm/vllm-openai:latest
          args:
            - "--model"
```

```
            - "Qwen/Qwen3-0.6B"
            - "--dtype"
            - "half"
            - "--seed"
            - "0"
            - "--gpu-memory-utilization"
            - "0.70"
            - "--max-model-len"
            - "1024"
            - "--host"
            - "0.0.0.0"
            - "--port"
            - "8000"
          ports:
            - containerPort: 8000
          resources:
            limits:
              nvidia.com/gpu: 1
              cpu: "4"
              memory: "12Gi"
          volumeMounts:
            - name: hf-cache
              mountPath: /root/.cache/huggingface
      volumes:
        - name: hf-cache
          emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: vllm-svc
spec:
  selector:
    app: vllm
  ports:
    - name: http
      port: 8000
      targetPort: 8000
```

Why these args are justified from primary docs:

- `--model`, `--seed`, `--gpu-memory-utilization`, and `--max-model-len` are documented engine args. [50]

Helm deployment option (useful if you want chart-managed conventions)

vLLM provides a Helm chart guide with explicit GPU resource configuration (`nvidia.com/gpu: 1`) and an example command to install the chart after setting values such as S3 download settings; the values table shows GPU limits/requests and image defaults. [51]

In a research repo, a practical pattern is:

- Commit your `values.yaml` with:
- pinned vLLM image tag
- explicit GPU and memory settings
- explicit vLLM engine arguments
- Use Helm to install/upgrade in a dedicated namespace.

NVIDIA Triton Inference Server on Kubernetes

Because Triton is a platform, your paper should clearly state which Triton LLM approach you benchmark. The two defensible options are:

- Triton + vLLM backend (engine comparable to vLLM standalone) [52]
- Triton + TensorRT-LLM backend (engine-level alternative to vLLM) [8]

Also, pin Triton tags based on driver compatibility. Triton publishes a **release compatibility matrix** mapping container tags to required CUDA driver versions; you should choose a tag whose required driver version is compatible with the driver on your laptop. [53]

Path A: Triton with vLLM backend (quick bring-up, comparable engine)

Triton's official "Deploying a vLLM model in Triton" tutorial provides an exact model repository structure and shows:

- `model_repository/vllm_model/1/model.json`
- `model_repository/vllm_model/config.pbtxt`
- `model.json` can control `gpu_memory_utilization` (example sets 0.5); it also notes vLLM defaults to aggressively consuming GPU memory unless you cap it. [28]

It also documents Triton's readiness ports: HTTP at 8000, gRPC at 8001, metrics at 8002 (matching Triton metrics docs). [54]

Model packaging (Triton vLLM backend)
Follow the upstream sample and commit the generated files into your repo so the benchmark is fully reproducible. The tutorial shows exactly how to download the sample `model.json` and `config.pbtxt`. [28]

Kubernetes deployment (single GPU, hostPath or PVC)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: triton-vllm
spec:
  replicas: 1
  selector:
    matchLabels:
      app: triton-vllm
  template:
    metadata:
      labels:
        app: triton-vllm
    spec:
      containers:
        - name: triton
          image: nvcr.io/nvidia/tritonserver:24.07-vllm-python-py3
          args:
            - "tritonserver"
            - "--model-store=/models"
          ports:
            - { name: http, containerPort: 8000 }
            - { name: grpc, containerPort: 8001 }
            - { name: metrics, containerPort: 8002 }
          resources:
            limits:
              nvidia.com/gpu: 1
              cpu: "4"
              memory: "12Gi"
          volumeMounts:
            - name: model-repo
              mountPath: /models
      volumes:
        - name: model-repo
          hostPath:
            path: /absolute/path/on/host/model_repository
            type: Directory
---
apiVersion: v1
kind: Service
metadata:
  name: triton-vllm-svc
spec:
  selector:
    app: triton-vllm
  ports:
    - { name: http, port: 8000, targetPort: 8000 }
```

```
    - { name: grpc, port: 8001, targetPort: 8001 }
    - { name: metrics, port: 8002, targetPort: 8002 }
```

For the paper, document Triton's `/metrics` endpoint (default 8002) and whether you disable/enable GPU metrics and the polling interval (`--metrics-interval-ms`). [55]

Request interface for benchmarking
The vLLM backend tutorial shows using the `/v2/models/vllm_model/generate` endpoint and provides the JSON structure. [28]

Path B: Triton with TensorRT-LLM backend (engine-level alternative)

This path is heavier but scientifically valuable if your goal is to compare a "PyTorch/vLLM serving engine" vs "TensorRT-LLM engine served by Triton."

The TensorRT-LLM backend docs:

- Specify the relevant container tag pattern `nvcr.io/nvidia/tritonserver:<xx.yy>-trtllm-python-py3`. [56]
- Provide a quick-start engine build flow using a small model (`gpt2-medium`) and `trtllm-build`, and emphasize aligning backend versions with a support matrix. [57]
- Define numerous scheduler/batching knobs in the model config docs (e.g., `triton_backend`, `triton_max_batch_size`, `batching_strategy`, `max_queue_delay_microseconds`, queue sizing, KV cache fractions). [58]

Practical way to do this on a laptop
Split it into two Kubernetes Jobs/Deployments:

1) **Engine build Job**: runs the Triton TRTLLM container once, outputs engines to a PVC (or hostPath).
2) **Inference Deployment**: runs Triton server with the built engines mounted read-only.

This separation is consistent with the general TensorRT guidance that engine builds are time-consuming and usually done offline. [59]

At minimum, your paper should record:

- Triton container tag and driver requirement (from compatibility matrix) [53]
- The TensorRT-LLM build flags (e.g., max batch/tokens, kv cache type) [33]
- The Triton TensorRT-LLM backend config (queue delay, batching strategy) [58]

Ray Serve LLM on Kubernetes (KubeRay)

Ray's official Kubernetes example for Ray Serve LLM:

- Deploys a RayService that uses `ray.serve.llm:build_openai_app` with `engine_kwargs` including `gpu_memory_utilization` and `max_model_len`. [60]

- Uses a container image `rayproject/ray-llm:...-cu...` and sets GPU resources and Ray `rayStartParams` accordingly in its sample YAML. [61]
- Requires KubeRay operator installation and documents a standard workflow: install operator, apply RayService, port-forward dashboard. [62]

Install KubeRay operator (Helm)

KubeRay Helm charts repo provides the exact `helm repo add` command and install invocation. [63]

Single-GPU RayService YAML (adapted from upstream)

The upstream sample YAML is configured for 4 GPUs and large CPU/memory; for a laptop study you should scale it down to 1 GPU and modest CPU/memory. The raw upstream config clearly shows where to edit `nvidia.com/gpu` and `num-gpus`. [64]

A simplified conceptual schema (you will implement as a checked-in YAML in your repo):

- Set worker `resources.limits.nvidia.com/gpu: "1"`
- Set worker `rayStartParams.num-gpus: "1"`
- Set `engine_kwargs.dtype` to `half` or `float16` if BF16 is problematic on your stack; Ray's example uses BF16, but your GPU/software combo may dictate otherwise. [65]
- Use a small model first (`Qwen2.5-0.5B` or `Qwen3-0.6B`) and cap `max_model_len` (e.g., 1024). The Ray docs show how `engine_kwargs` map to vLLM settings. [66]

Ray provides built-in metrics in Prometheus format and documents the dashboard and monitoring features, including the fact that Ray does not ship a storage backend for metrics and expects you to integrate with Prometheus/Grafana (KubeRay docs include instructions). [67]

Benchmark methodology, experiment matrix, and paper execution plan

Architecture diagram for your benchmark environment

```
flowchart LR
  subgraph K8s["Kubernetes (single-node, 1x RTX 3060)"]
    subgraph GPUStack["GPU enablement"]
      DP["NVIDIA Device Plugin / GPU Operator\nexposes nvidia.com/gpu"]
      CTK["NVIDIA Container Toolkit\n(Docker/containerd runtime config)"]
    end

    subgraph Systems["Inference systems under test"]
      VLLM["vLLM Deployment\n(OpenAI-compatible /metrics)"]
      TRV["Triton + vLLM backend\n(/generate, :8002/metrics)"]
      TRT["Triton + TensorRT-LLM backend\n(/generate, :8002/metrics)"]
      RAY["RayService (Ray Serve LLM)\n(OpenAI ingress + vLLM engine)"]
    end
```

```
  MON["Prometheus + DCGM Exporter\n(optional Grafana)"]
end

LG["Load generator\n(wrk / locust / asyncio clients)"] --> VLLM
LG --> TRV
LG --> TRT
LG --> RAY

VLLM --> MON
TRV --> MON
TRT --> MON
RAY --> MON
```

This architecture is grounded in the fact that vLLM and Triton expose Prometheus-style metrics endpoints (`/metrics`), and DCGM Exporter exposes GPU metrics for Prometheus scraping. [68]

Workload design: what you should vary and why

A publishable benchmark is not "one concurrency value, one prompt." You want a small set of controlled sweeps that reveal throughput–latency tradeoffs and memory behavior.

A defensible minimum set of workloads:

- **Interactive**: short prompt (e.g., 64–128 input tokens), short output cap (e.g., 32–64 output tokens), concurrency sweep. This reveals platform overhead and router costs.
- **Summarization / long prefill**: long prompt (e.g., 512–1024 input tokens), moderate output (128–256). This stresses prefill and KV cache. It aligns with the observation that longer sequences magnify system-level bottlenecks and memory management issues in LLM serving. [69]
- **Decode-heavy generation**: short prompt, long output (e.g., 256+). This stresses decoding throughput.

Traffic patterns:

- Use both **saturation runs** ("send as fast as possible") and **rate-controlled runs** to find a stable operating point. vLLM's benchmark tooling explicitly supports request-rate controls and burstiness/ramp-up patterns. [70]

Metrics: what to report in the paper

You want both **service-level** and **system-level** metrics.

Service-level metrics (per request / per token)
Report distributions (p50/p90/p99) for at least:

- End-to-end latency (request → full completion)
- Time-to-first-token (TTFT) if streaming is enabled (or approximate via server/client side timestamps)

- Output tokens/sec (or total tokens/sec) and request throughput (req/s)
- Error rate (HTTP 4xx/5xx, timeouts)

System-level metrics (capacity and contention)

- GPU utilization and GPU memory usage over time
- CPU usage (for tokenization/postprocessing overhead)
- Queue depth / scheduler metrics where available:
- vLLM exposes scheduler/request counters via Prometheus metrics. [71]
- Triton exposes Prometheus metrics at `:8002/metrics` and includes both request and GPU metrics (with flags to enable/disable GPU metrics). [55]
- Ray Serve exposes request/latency metrics in Prometheus format and documents histogram bucket behavior. [72]
- DCGM Exporter exposes GPU metrics for Prometheus scraping. [73]

Cost proxy metrics (for a single laptop GPU, you can still compute an abstraction):

- GPU-seconds per 1K output tokens (lower is “cheaper”)
- GPU memory headroom at target p99 latency (operational viability)
- If you later extend to cloud, map GPU-seconds to actual $/hour pricing (outside the laptop-only scope).

Reproducibility checklist (what your paper should include)

Your results will be taken more seriously if every figure is reproducible from a repo tag.

Required:

- Exact container image tags (and preferably digests) for:
- vLLM Docker image [74]
- Triton server tag(s) and their driver/CUDA requirements (compatibility matrix) [53]
- Ray Serve LLM image from the sample RayService (or your pinned derivative) [64]
- Exact Kubernetes distribution and version (minikube/microk8s/k3s paths differ in GPU plumbing). [75]
- GPU driver version (because Triton tags bind to CUDA driver requirements). [53]
- Fixed seeds for sampling: vLLM documents `--seed` and why global seeding matters for reproducibility. [25]
- Fixed model revisions if possible (vLLM supports explicit HF revisions). [76]
- A committed workload file: prompts, target input token bins, output caps, concurrency levels, run durations.

Experiment matrix

This table is a template for “what varies.” Your actual paper can subset it if runtime is limited, but the matrix shows reviewers you understood the space.

Dimension	Levels (recommended laptop study)	Notes / mapping
System under test	vLLM standalone; Triton+vLLM backend; Ray Serve LLM (vLLM engine); Triton+TensorRT-LLM backend	Triton supports both a vLLM backend and TensorRT-LLM backend. [77]
Model tier	opt-125m (sanity); 0.5–0.6B (small); 1–4B (medium if VRAM allows); 7B (stress, likely quantized/short context)	opt-125m is used in official tutorials (vLLM stack and Triton vLLM tutorial). [78]
Precision / quantization	FP16; INT8; INT4 (AWQ / bitsandbytes)	vLLM lists supported quantization methods and hardware applicability. [26]
Framework backend	PyTorch (vLLM); TensorRT-LLM engines via Triton; (optional) ONNX/TensorRT via Triton for non-LLM baselines	Triton supports multiple frameworks; TRT-LLM backend is explicitly supported for LLMs. [79]
Max model length	512; 1024; 2048 (only if VRAM permits)	vLLM documents `--max-model-len` (incl. auto-fitting) and memory utilization control. [80]
Input length (tokens)	64; 256; 512; 1024	Stress prefill and KV memory. [69]
Output cap (tokens)	32; 128; 256	Stress decoding throughput.
Concurrency	1; 2; 4; 8; 16; 32 (increase until OOM or p99 blowup)	vLLM benchmarking tools explicitly model concurrency and request rate. [70]
Batch behavior controls	vLLM: implicit batching; Triton TRT-LLM: queue delay, batching strategy; Ray: max_ongoing_requests, replicas	TRT-LLM backend exposes queue and batching strategy knobs. [81]

Suggested plots (what figures to include)

A minimal “results” section should have:

- **Latency–throughput curves** per system at fixed model + token lengths (plot throughput vs p99 latency). This is consistent with LLM serving tradeoff analysis practices in the literature. [11]
- **p50/p99 latency vs concurrency** at fixed input/output lengths.
- **GPU utilization and memory over time** during a step-load or ramp-up run (show stability, throttling, or OOM). Use DCGM exporter and server `/metrics`. [82]
- **Error rate vs concurrency** (including OOM and timeout behaviors).

Data collection and logging: practical methods

Fastest "works everywhere" approach (good enough for a laptop paper draft):

- Your load generator records per-request timestamps and parses token usage from responses (where available). Ray's example shows OpenAI-like responses include token usage fields; vLLM and Ray Serve LLM are OpenAI-compatible. [83]
- Periodically scrape each system's `/metrics` endpoint to a time-stamped text file.
- Record `nvidia-smi` samples in parallel (if you prefer not to deploy Prometheus locally).

More rigorous, Kubernetes-native approach (better for a final paper):

- Deploy **kube-prometheus-stack** (Triton's helm deployment guide explicitly uses it to collect and display metrics) and add ServiceMonitors/PodMonitors for each server. [84]
- Deploy **DCGM Exporter** for GPU metrics. [73]
- For Ray Serve, follow KubeRay's Prometheus/Grafana guidance to scrape Ray metrics. [85]

Load generation scripts and metric scraping: pseudocode

Async load generator for OpenAI-compatible endpoints (vLLM standalone, Ray Serve LLM)

vLLM and Ray Serve LLM provide OpenAI-compatible APIs; vLLM explicitly notes OpenAI client compatibility. [86]

```
# pseudo_load_openai_async.py
import asyncio, time, json
import httpx

PROMPTS = [...]  # committed prompt set with known token bins
OUT_TOKENS = 128

async def one_request(client, url, model, prompt):
    t0 = time.perf_counter()
    payload = {
        "model": model,
        "prompt": prompt,              # use /v1/completions-style payload for
maximal model compatibility
        "max_tokens": OUT_TOKENS,
        "temperature": 0.0,
    }
    r = await client.post(url, json=payload, timeout=120)
    t1 = time.perf_counter()

    ok = (r.status_code == 200)
    data = r.json() if ok else {}
    usage = data.get("usage", {})     # OpenAI-compatible servers often return
usage fields
```

```
    return {
        "ok": ok,
        "status": r.status_code,
        "latency_s": t1 - t0,
        "prompt_len_chars": len(prompt),
        "usage": usage,
    }

async def run_concurrency(concurrency, duration_s, base_url):
    async with httpx.AsyncClient() as client:
        t_end = time.time() + duration_s
        inflight = set()
        results = []

        while time.time() < t_end:
            while len(inflight) < concurrency:
                prompt = pick_prompt(PROMPTS)  # deterministic sampling seeded
in your harness
                task = asyncio.create_task(one_request(
                    client,
                    f"{base_url}/v1/completions",
                    model="qwen3-0.6b",  # match your deployment
                    prompt=prompt,
                ))
                inflight.add(task)

            done, inflight = await asyncio.wait(inflight,
return_when=asyncio.FIRST_COMPLETED)
            for d in done:
                results.append(d.result())

        return results

# Write JSONL -> analyze with pandas later
```

Notes you should document in the paper:

- Use deterministic sampling seeded at the harness level, and also server-side seeds where supported (vLLM `--seed`). [25]
- Include warm-up (discard first N requests).

HTTP load generator for Triton vLLM-backend `/generate`

The official Triton vLLM backend tutorial uses `/v2/models/vllm_model/generate` and shows the JSON schema (`text_input`, `parameters`). [28]

```python
# pseudo_load_triton_generate_async.py
import asyncio, time, json
import httpx

async def one_triton_generate(client, url, prompt):
    t0 = time.perf_counter()
    payload = {
        "text_input": prompt,
        "parameters": {"stream": False, "temperature": 0.0, "max_tokens": 128},
    }
    r = await client.post(url, json=payload, timeout=120)
    t1 = time.perf_counter()
    return {"ok": r.status_code == 200, "status": r.status_code, "latency_s":
t1 - t0}

async def run(concurrency, n_requests, base_url):
    url = f"{base_url}/v2/models/vllm_model/generate"
    async with httpx.AsyncClient() as client:
        sem = asyncio.Semaphore(concurrency)

        async def wrapped(prompt):
            async with sem:
                return await one_triton_generate(client, url, prompt)

        tasks = [asyncio.create_task(wrapped(PROMPTS[i % len(PROMPTS)])) for i
in range(n_requests)]
        return await asyncio.gather(*tasks)
```

Metrics scraping (Prometheus text format)

- Triton metrics are available at `:8002/metrics` by default. [55]
- vLLM metrics are exposed at `/metrics` on the OpenAI server. [31]
- DCGM exporter exposes GPU metrics at `/metrics`. [87]

```python
# pseudo_scrape_metrics.py
import time, requests

TARGETS = {
  "vllm": "http://vllm-svc:8000/metrics",
  "triton": "http://triton-vllm-svc:8002/metrics",
  "dcgm": "http://dcgm-exporter:9400/metrics",
}

def scrape_loop(out_dir, interval_s=5):
    while True:
        ts = int(time.time())
```

```
        for name, url in TARGETS.items():
            try:
                text = requests.get(url, timeout=5).text
                open(f"{out_dir}/{name}_{ts}.prom", "w").write(text)
            except Exception as e:
                open(f"{out_dir}/{name}_{ts}.err", "w").write(str(e))
        time.sleep(interval_s)
```

Using built-in benchmark tools where appropriate

You can strengthen credibility by triangulating results with vendor/official benchmarking tools:

- vLLM provides a benchmark CLI and documents parameters for request rate, burstiness, and max concurrency; use it to cross-check your custom harness. [70]
- Triton provides **Perf Analyzer** documentation and describes it as a load generator that measures latency/throughput; it also has LLM-specific guidance in its docs tree. [88]

Timeline and paper production plan

A realistic timeline for a laptop-based study should include explicit “bring-up” and “measurement validity” phases.

```
gantt
  title Kubernetes LLM Inference Benchmark Paper Timeline
  dateFormat  YYYY-MM-DD
  axisFormat  %b %d

  section Environment
  GPU drivers + container toolkit verified      :a1, 2026-03-14, 3d
  Local Kubernetes w/ nvidia.com/gpu working    :a2, after a1, 3d

  section Deployments
  vLLM baseline deployment + metrics            :b1, after a2, 2d
  Triton vLLM backend deployment + metrics      :b2, after b1, 2d
  Ray Serve LLM (KubeRay) deployment + metrics  :b3, after b2, 3d
  (Optional) Triton TensorRT-LLM path           :b4, after b2, 5d

  section Benchmark design
  Workload suite + prompt set committed         :c1, after b1, 2d
  Load generator + result schema                :c2, after c1, 3d
  Monitoring (Prometheus + DCGM exporter)       :c3, after c1, 3d

  section Experiments
  Concurrency sweeps (small model tier)         :d1, after c2, 4d
  Memory stress + max_model_len sweeps          :d2, after d1, 3d
  Quantization sweeps (if feasible)             :d3, after d2, 4d
```

```
section Writing
Methodology + threats to validity            :e1, after c2, 3d
Results + discussion                         :e2, after d3, 5d
Repro artifacts + final edit                 :e3, after e2, 3d
```

Limitations on a single RTX 3060 and pragmatic alternatives

Your hardware constraint is an advantage (realistic "edge/laptop" setting) but limits generality:

- You cannot study multi-node scaling, multi-GPU tensor parallelism, or large-context workloads at high concurrency with large models; those are fundamentally governed by VRAM and GPU count, and serve as threats to validity you should explicitly state. [69]
- TensorRT-LLM engine build workflows can be heavy; NVIDIA's own examples demonstrate multi-GPU flags and large engine output setups, so you should plan to start with small models and minimal batch sizes on a laptop. [89]

Alternatives you can document and (optionally) extend to:

- Use **quantization** (vLLM supports multiple quant formats including AutoAWQ and BitsAndBytes). [26]
- Use **shorter contexts** (`--max-model-len`) and cap GPU allocation (`--gpu-memory-utilization`). [80]
- If you later add cloud experiments, keep the same Kubernetes manifests and workload files; Triton's own helm deployment guidance emphasizes matching cluster GPU drivers/CUDA versions to the Triton release you use. [90]

Learning resources prioritized for this project

Primary/official sources most aligned with the work (recommended reading order):

- vLLM core design + serving:
- vLLM / PagedAttention paper [6]
- vLLM OpenAI-compatible server docs [91]
- vLLM engine args (memory utilization, max model len, seed) [92]
- vLLM metrics docs [47]
- vLLM quantization docs [26]
- Triton and TensorRT-LLM:
- Triton user guide + metrics endpoint [93]
- Triton vLLM backend tutorial (model repo, `/generate`) [28]
- Triton TensorRT-LLM backend docs + model config parameters [8]
- Triton release compatibility matrix (choose tags that match your driver) [53]
- TensorRT-LLM numerical precision docs [27]

- Ray Serve LLM on Kubernetes:

- Ray Serve LLM Kubernetes deployment guide/example [60]
- Ray Serve LLM vLLM compatibility (engine_kwargs parity) [94]
- Ray Serve monitoring/metrics [72]
- KubeRay Helm charts (operator install) [63]

- Ray GPU usage on Kubernetes (how GPU limits map to Ray scheduling) [95]

- Kubernetes GPU fundamentals and GPU enablement:

- Kubernetes “Schedule GPUs” and extended resource semantics [35]
- Kubernetes device plugin framework [96]
- NVIDIA Container Toolkit install/config (Docker and containerd) [16]
- NVIDIA GPU Operator getting started and deployment scenarios [97]
- DCGM exporter docs for GPU telemetry [73]

Methodology/reference literature to cite in your paper’s related-work / motivation:

- OSDI’24 throughput–latency tradeoff analysis for LLM serving systems (includes vLLM comparisons). [11]

---

[1] [4] [6] [69] Efficient Memory Management for Large Language Model Serving with PagedAttention
https://arxiv.org/abs/2309.06180?utm_source=chatgpt.com

[2] [8] [33] [56] [57] [89] https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/tensorrtllm_backend/README.html
https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/tensorrtllm_backend/README.html

[3] [32] [60] [65] [83] https://docs.ray.io/en/latest/cluster/kubernetes/examples/rayserve-llm-example.html
https://docs.ray.io/en/latest/cluster/kubernetes/examples/rayserve-llm-example.html

[5] [9] [28] [54] [78] https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/tutorials/Quick_Deploy/vLLM/README.html
https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/tutorials/Quick_Deploy/vLLM/README.html

[7] [79] [93] https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/index.html
https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/index.html

[10] [66] [94] https://docs.ray.io/en/latest/serve/llm/user-guides/vllm-compatibility.html
https://docs.ray.io/en/latest/serve/llm/user-guides/vllm-compatibility.html

[11] https://www.usenix.org/system/files/osdi24-agrawal.pdf
https://www.usenix.org/system/files/osdi24-agrawal.pdf

[12] [31] [47] [68] https://docs.vllm.ai/en/v0.6.6/serving/metrics.html
https://docs.vllm.ai/en/v0.6.6/serving/metrics.html

[13] [23] [24] [25] [30] [48] [50] [76] [80] [92] https://docs.vllm.ai/en/stable/configuration/engine_args/
https://docs.vllm.ai/en/stable/configuration/engine_args/

14 72 https://docs.ray.io/en/latest/serve/monitoring.html
https://docs.ray.io/en/latest/serve/monitoring.html

15 38 39 40 75 https://minikube.sigs.k8s.io/docs/tutorials/nvidia/
https://minikube.sigs.k8s.io/docs/tutorials/nvidia/

16 Installing the NVIDIA Container Toolkit
https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html?utm_source=chatgpt.com

17 35 https://kubernetes.io/docs/tasks/manage-gpus/scheduling-gpus/
https://kubernetes.io/docs/tasks/manage-gpus/scheduling-gpus/

18 84 90 https://github.com/triton-inference-server/server/blob/main/deploy/gcp/README.md
https://github.com/triton-inference-server/server/blob/main/deploy/gcp/README.md

19 86 https://docs.vllm.ai/en/latest/serving/openai_compatible_server.html
https://docs.vllm.ai/en/latest/serving/openai_compatible_server.html

20 https://github.com/triton-inference-server/client
https://github.com/triton-inference-server/client

21 62 https://docs.ray.io/en/latest/serve/production-guide/kubernetes.html
https://docs.ray.io/en/latest/serve/production-guide/kubernetes.html

22 https://huggingface.co/docs/transformers/v4.26.1/perf_train_gpu_one
https://huggingface.co/docs/transformers/v4.26.1/perf_train_gpu_one

26 https://docs.vllm.ai/en/latest/features/quantization/
https://docs.vllm.ai/en/latest/features/quantization/

27 https://nvidia.github.io/TensorRT-LLM/reference/precision.html
https://nvidia.github.io/TensorRT-LLM/reference/precision.html

29 https://docs.vllm.ai/en/latest/deployment/integrations/production-stack/
https://docs.vllm.ai/en/latest/deployment/integrations/production-stack/

34 https://github.com/NVIDIA/TensorRT-LLM/blob/main/examples/models/core/qwen/README.md
https://github.com/NVIDIA/TensorRT-LLM/blob/main/examples/models/core/qwen/README.md

36 NVIDIA device plugin for Kubernetes
https://github.com/NVIDIA/k8s-device-plugin?utm_source=chatgpt.com

37 NVIDIA/gpu-operator
https://github.com/NVIDIA/gpu-operator?utm_source=chatgpt.com

41 Add-on: gpu - MicroK8s
https://canonical.com/microk8s/docs/addon-gpu?utm_source=chatgpt.com

42 https://docs.k3s.io/advanced
https://docs.k3s.io/advanced

43 RayCluster Quickstart — Ray 2.54.0
https://docs.ray.io/en/latest/cluster/kubernetes/getting-started/raycluster-quick-start.html?utm_source=chatgpt.com

44 Minikube versus Kind: GPU Support : r/kubernetes
https://www.reddit.com/r/kubernetes/comments/1ilb8v2/minikube_versus_kind_gpu_support/?utm_source=chatgpt.com

45 https://canonical.com/microk8s/docs/addon-gpu
https://canonical.com/microk8s/docs/addon-gpu

46 74 https://docs.vllm.ai/en/stable/deployment/docker/
https://docs.vllm.ai/en/stable/deployment/docker/

49 https://hub.docker.com/r/vllm/vllm-openai/tags
https://hub.docker.com/r/vllm/vllm-openai/tags

51 https://docs.vllm.ai/en/stable/deployment/frameworks/helm/
https://docs.vllm.ai/en/stable/deployment/frameworks/helm/

52 77 https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/vllm_backend/README.html
https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/vllm_backend/README.html

53 https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/introduction/compatibility.html
https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/introduction/compatibility.html

55 https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/metrics.html
https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/metrics.html

58 81 https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/tensorrtllm_backend/docs/model_config.html
https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/tensorrtllm_backend/docs/model_config.html

59 https://docs.nvidia.com/deeplearning/tensorrt/latest/getting-started/quick-start-guide.html
https://docs.nvidia.com/deeplearning/tensorrt/latest/getting-started/quick-start-guide.html

61 64 https://raw.githubusercontent.com/ray-project/kuberay/master/ray-operator/config/samples/ray-service.llm-serve.yaml
https://raw.githubusercontent.com/ray-project/kuberay/master/ray-operator/config/samples/ray-service.llm-serve.yaml

63 https://github.com/ray-project/kuberay-helm
https://github.com/ray-project/kuberay-helm

67 https://docs.ray.io/en/latest/cluster/metrics.html
https://docs.ray.io/en/latest/cluster/metrics.html

70 https://docs.vllm.ai/en/latest/benchmarking/cli/
https://docs.vllm.ai/en/latest/benchmarking/cli/

71 https://docs.vllm.ai/en/stable/design/metrics/
https://docs.vllm.ai/en/stable/design/metrics/

73 82 87 https://docs.nvidia.com/datacenter/cloud-native/gpu-telemetry/latest/dcgm-exporter.html
https://docs.nvidia.com/datacenter/cloud-native/gpu-telemetry/latest/dcgm-exporter.html

85 https://docs.ray.io/en/latest/cluster/kubernetes/k8s-ecosystem/prometheus-grafana.html
https://docs.ray.io/en/latest/cluster/kubernetes/k8s-ecosystem/prometheus-grafana.html

88 https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/perf_analyzer/docs/README.html
https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/perf_analyzer/docs/README.html

91 OpenAI-Compatible Server - vLLM
https://docs.vllm.ai/en/latest/serving/openai_compatible_server.html?utm_source=chatgpt.com

95 https://docs.ray.io/en/latest/cluster/kubernetes/user-guides/gpu.html
https://docs.ray.io/en/latest/cluster/kubernetes/user-guides/gpu.html

96 https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/device-plugins/
https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/device-plugins/

97 Installing the NVIDIA GPU Operator
https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/getting-started.html?utm_source=chatgpt.com